Registry transparency: completeness proofs for the public fabric

TBK SPEC 41:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 41:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. The property nobody has 4

2. The canonical snapshot 5

2.1 Where each column comes from (SSOT, never a re-derivation) 5

3. The tree 6

4. The log head and the chain law 7

5. The verbs 8

6. The tombstone law 8

7. Named errors and exit codes 9

8. What this spec does not own 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Status: PLANNED (the design is locked here first; §5's verbs ship together with this spec in the same change).

1. The property nobody has

A signature over a registry proves that whoever holds the key wrote *those bytes*. It says nothing about what was there last month. Yanking a row and re-signing is indistinguishable from the row never existing, and that is the one attack a mirror cannot detect: every mirror is happy to serve the smaller file.

Completeness is the missing property, and it needs machinery rather than good intentions:

- **append-only**: the registry's history is a log that only grows;
- **provable**: one row's presence is provable without shipping the log;
- **auditable**: a row that disappeared without saying so is a NAMED alarm, not a diff somebody has to notice.

The machinery is an RFC 6962-shaped Merkle log (Certificate Transparency's shape) because it is the one construction third-party tooling already implements: inclusion proofs of `log2(n)` hashes, and a chain of heads that proves the log only grew.

© 2026 The Tebako Project

2. The canonical snapshot

`tebako registry snapshot <alias|path>` renders every row of a registry document as ONE canonical line. Determinism is the premise — a snapshot that shifts when nothing changed is worthless as evidence:

FIGURE 1

```
<kind> <name>@<version> <platform> <sha256|-> <state>
<keyid|->
```

TABLE 1

field	rule
`kind`	the document's `kind:` value, verbatim
`name@version`	verbatim from the document (one line per row, so a version entry with N platform legs is N lines)
`platform`	the triplet, or `universal`; a variant arm renders `<variant-id>:<universal|triplet>` — the arm id never stands alone, so a universal arm can never be mistaken for a triplet
`sha256`	the row's anchor digest, lowercase hex; `-` when the row declares none (a runtime index row)
`state`	`active`, or `withdrawn` — spec 04's `status:` value verbatim, never a second spelling (§6)
`keyid`	the row's declared signer keyid when present, else `-`

Order: `(kind, name, version, platform)` — byte order, never document order. No timestamps, host names, comments or map iteration order enter a line: an author may reorder the YAML, rewrite its comments, or re-indent it, and the snapshot does not move. Two authors producing the same set of rows produce the same snapshot.

2.1 Where each column comes from (SSOT, never a re-derivation)

- `platform` — a `variants:` arm renders its id from `tebako_resolve::registry::variant_id()` (spec 28 §2's one derivation). A second implementation of that derivation would produce snapshots that disagree with resolution, which is the one thing a transparency log cannot survive.
- `sha256` — the platform row's own `sha256`. A `platforms: universal` row carries NO digest column in the registry model (`ReleaseRef` holds a reference, not a hash; the anchor is the release's `SHA256SUMS` plus its attestation). Such a row renders `-`, and that is the honest value: this log proves the ROW SET is complete, while the release's own

© 2026 The Tebako Project

sums and attestations prove the BYTES. A `?sha256=` pin inside the ref, when present, is rendered as that pin.

- `state` — spec 04's `status`: verbatim (`withdrawn` when yanked), `active` when absent. Never a second vocabulary (§6).
- `keyid` — the row's declared signer (`signature.keyid`) when present, else `-`.

`--json` emits the `registry_log_schema: 1` document:

FIGURE 2

```
{"registry_log_schema": 1, "registry": "official", "rows":
412,
 "lines": ["…"], "root": "…", "head_hash": "…"}
```

3. The tree

Leaves and nodes are domain-separated, so a leaf hash can never be mistaken for an interior node:

FIGURE 3

```
leaf(line_i)  = SHA-256(0x00 ‖ line_i bytes)
node(l, r)    = SHA-256(0x01 ‖ l ‖ r)
empty         = SHA-256("")                    (a registry
with no rows)
```

The fold is RFC 6962's: the largest power of two strictly smaller than the leaf count splits the tree, recursively. An odd tail is carried up, never padded — padding is what breaks third-party verifiers.

Inclusion proof for leaf `i` of n: the sibling hashes bottom-up, serialized as a JSON array of lowercase hex, plus the leaf's index and the tree's leaf count. A verifier recomputes the root from the line and the proof; nothing else is needed.

© 2026 The Tebako Project

4. The log head and the chain law

Each snapshot is appended to the registry's log as one head record:

FIGURE 4

```
<seq> ts=<unix> root=<hex> rows=<n> prev=<hex|-> key=<keyid>
sig=<hex|->
```

- `seq` — 0, 1, 2, … ; strictly increasing, no gaps (a gap is a named error: the log was edited).
- `prev` — the previous head's `root` (`-` for `seq 0`). A head whose `prev` does not match the previous head's `root` is a **fork**, named as such: the log diverged and nothing after that point can be trusted.
- `rows` — the snapshot's leaf count; monotonic non-decreasing unless tombstones account for the difference (§6).
- `sig` — detached OpenPGP over

  FIGURE 5

  ```
  tebako-registry-log-1\n<seq>\n<ts>\n<root>\n<rows>\n<prev>
  \n
  ```

  signed by the registry's declared head key (spec 09 §9.1's `signing:` block — the key the registry already carries; no new key class). The signature is verified against the pinned trust store, and an unknown signer is `unknown-key` (reported, never upgraded to trusted).

The log file is a head record per line, in `seq` order:

FIGURE 6

```
<registry>.log          (beside the registry it describes)
```


© 2026 The Tebako Project

5. The verbs

FIGURE 7

```
tebako registry snapshot <alias|path> [--json] [--out <path>]
tebako registry log verify <log> [--registry <path>] [--json]
tebako registry log prove <log> --registry <path> --row
<name>@<version>
              [--json] [--verify-root <hex>]
```

- `snapshot` renders §2's lines and root. `--out` writes them beside the registry for publication; without it they go to stdout (`--json` emits the document).
- `log verify` checks, in order: every head's signature; the `seq` continuity; the `prev` chain (a fork is named at the head where it happens); and — when `--registry` is given — whether the document IS the row set the newest head sealed, by RECOMPUTING THE ROOT from the document's canonical rows. One number answers both halves: if the roots agree, no row could have been added, removed or edited without moving it. A `rows` decrease is reported separately (the chain alone cannot see tombstones), and a forced deletion therefore surfaces by name.
- `log prove` emits the inclusion proof for `--row` against the newest head, and one proof per matching row: a version is often several rows (one per platform), and hiding all but one would understate what the registry says. It needs `--registry` as well as `--row`, because the canonical line cannot be reconstructed from a name alone — the line IS what is proved. `--verify-root <hex>` checks the proof against a root the caller obtained out of band: a user holding ONE 32-byte value can prove a row's membership without trusting the download that carried it.

Output discipline follows the toolchain's rule: `--json` documents on stdout alone (banner to stderr), human lines on stdout otherwise, exit codes per §7.

6. The tombstone law

A row may leave a registry; it may not leave *silently*. The vocabulary for leaving already exists and this spec does not invent a second one: spec 04's `status: withdrawn` (roadmap 85) is the tombstone — the resolver refuses a withdrawn row by name (`WithdrawnPayload`), listing surfaces render it as withdrawn, and release assets stay immutable.

Under the completeness law the ROW STAYS: `tebako registry retire` sets `status: withdrawn` on the version instead of deleting it, so the next snapshot renders the row as

© 2026 The Tebako Project

`withdrawn` and keeps it in the tree forever. A registry whose rows only ever disappear is a registry nobody can audit.

Withdrawal is the only legal way for a name to vanish from a snapshot or for `rows` to fall. `--force` still removes a row outright — an operator may need it — but the next `log verify` reports the disappearance by row name, and a forced removal is precisely the alarm the log exists to raise: never quiet.

7. Named errors and exit codes

TABLE 2

code	class	meaning
71	`EX_TEBAKO_SIGNATURE`	a head's signature does not validate
72	`EX_TEBAKO_TRUST`	a head's signer key is not in the trust store (`unknown-key` in JSON; the chain is still reported)
80	`EX_TEBAKO_LEDGER`	an integrity finding: a `seq` gap, a fork, a `prev` mismatch, a proof that does not verify, a row missing without a tombstone, or a tree/file inconsistency (spec 40 §6's ledger class — both streams are record streams whose integrity findings are one kind of event)
74	`EX_TEBAKO_IO`	a log or registry cannot be read or written
64	`EX_USAGE`	unknown flag, unparseable row reference, `--verify-proof` without a proof source

Every message names the row, the head, or the file — never "invalid input".

8. What this spec does not own

- Registry document grammar and the `signing:` block: spec 04 and spec 37.
- The signature mechanism and the trust store: spec 09.
- Publication cadence and hosting: spec 13 and spec 37. This spec says what a published log must satisfy, never when to publish one.
- Rotation of the head key: spec 09 §11's successor statement — a log signed under a successor key chains through `prev` like any other head.

© 2026 The Tebako Project